Introduction to C Programming

Purpose of Programming.

- Purpose of programming is to make human life better and easier.
- It does so because the computer programs can perform a set of tasks once we define them clearly.
- Hence we can use a computer to solve our day to day problems by providing it clear instructions rather than solving it by yourself.
- This saves a lot of human time and energy so that we can concentrate on other challenging tasks that cannot be automated.

Example

- To run a supermarket business, rather than calculating bill manually every time, we can develop a program for billing.
- It would operate very fast so that customers need not wait in queue for a long time. We can also handle many customers easily. It increases the productivity.
- It would calculate bills with 100% accuracy rate, and we can even make the program generate automatic monthly reports, selling pattern, etc. for improving our business.

Let us see some day to day examples as well.

Dictionary Program

Instead of carrying a dictionary manually, we use a dictionary program which stores all the words with their meanings so that we can know any word's meaning from our computer or mobile very easily.

Calculator

Similarly, we use calculators in our mobiles/computers which are programmed to solve complex calculations in no time.

Search Engine

Search engines are another good examples. Google provides us search engine which acts as a face of the internet for us. Internally they read all the websites in the world and tag them so that we get desired information when we search for them from anywhere in the world.

Chat Application

Chat applications use computer networks to send and receive messages via mobile/internet and help us stay in touch with the people anywhere in the world.

Introduction to C Programming

- **C** is a general-purpose programming language which was invented by **Dennis Ritchie.**
- It was designed to implement **UNIX operating system**.
- In UNIX, both operating system and most of the programs are written in C.
- It is also called **system programming language** because it is useful for writing **system programs** (interface between the **hardware** and **user application**) like **compiler**, **device driver**, **BIOS** (Basic input/output system)**, linker, assembler etc.**

C is a portable programming language

- If you write a C code in your machine, it will run on any machine which supports C.
- Because it is not tied to any hardware or system. We can say, it is a hardware independent language or platform independent language.
- That is why C is called a **portable language.**

Basic Structure of C Program

- Basically, every C program has two major sections.

1. Header section **2. Main section**

1. Header Section (Why we use header files in C)

- Header files consist of multiple **pre-written codes**.
- **stdio.h** It will have all the information about i/o related functions definition.If our C program needs to say something to the user or get some input from the user, we should use **printf** and **scanf** function accordingly.
- whenever we need to use **input** and **output** related (ex. **printf/scanf**) **functions** in our program, we should include **stdio.h header file.**
- Likewise, **math.h** - will have all mathematical functions definition.
- Using **math.h header file** we can calculate, the **square root** of a number,**power**(x,y) i.e. xy etc.

Contd…

2. Main Section (Why main function is used in C)

- Every C program **must have a main function.**
- Main function is the **starting point** of the **program execution**.
- Wherever the main function is, it will be **called first**. So **main function** is the **heart of C program**.

Example

```
#include<stdio.h>      //header section

int main()             //main section
{
    return 0;
}
```

Hello World Program in C

Example:

```
#include<stdio.h>      //header section

int main()             //main section
{
    printf("Hello World");
    return 0;
}
```

Output:

Hello World

Contd…

What is **<stdio.h>** ?

- **stdio.h** is a header file which has the necessary information to include the **input/output related functions** in our program. Example **printf**, **scanf** etc.
- If we want to use **printf** or **scanf function** in our program, we should include the **stdio.h** header file in our source code.
- Otherwise, our program doesn't know what is the definition of printf or scanf and it will throw error/warning saying that implicit declaration of built-in function 'printf'

Why **#include** ?

- #include is a **preprocessor directive**.
- It will **include the file** which is given within the **angle brackets "<>"** into the **current source file.**

Example: If we use **#include<stdio.h>** in your c program, it will include **stdio.h file** into our **source program** which has the information for all **input**, **output** related functions.

Contd…

What is **int main()** ?

- As we discussed earlier, the main function is the **starting point of program execution**.
- **Operating system (OS)** initiates the **program execution** by **invoking** the **main function**.
- And it will **expect an integer value** from the main function. That integer value represents the **status of the program**. That's why we declared main function **return type as int.**
- **If any error** in the program while execution, the main function **will return some non-zero value** to the operating system.
- If the program **executed successfully** without any error, It will **return zero**.

Why ‘ **{** ‘ ?

- C is the **block structured programming language** where statements are **grouped together** to achieve specific task using **curly braces**.
- **Open curly brace {** represents the **starting point of the block.**

Contd…

printf (“ Hello World ”);

- **printf** is a function which is defined in **stdio.h header file.**
- It will **print everything to the screen** which is given **inside the double quotes**.
- In this case, **Hello World** will be printed as an **output**.
- **semicolon " ; "**
- Every statement in C **should end** with **semicolon ; .**

Contd…

Why **return 0;** ?

- As we declared main function **return type as an integer**, the calling function (operating system) will **expect an integer value** from the main function. In main function, we always **explicitly return 0.**
- Program execution will reach **return 0 statement** if and only if all the above statements are **executed without any error**.
- If program execution reaches **return 0 statement**, We can ensure that there is **no error in our program.**
- If some **error occurred** while program execution, the main function **will return some non-zero value** to the operating system to indicate the error.
- **Semicolon ;** indicates the **end of the statement.**

Why ‘ **}** ‘ ?

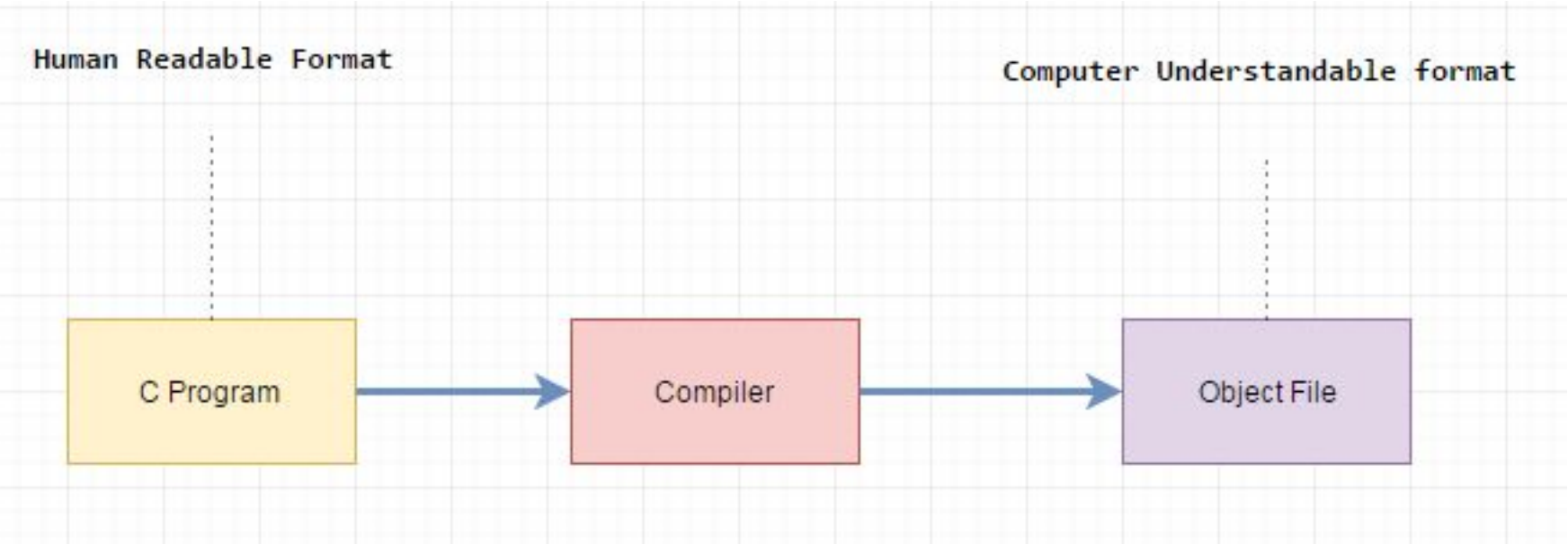
- **Close curly brace** indicates the **end of the block.**
- In other words, it indicates the **end of main function.**

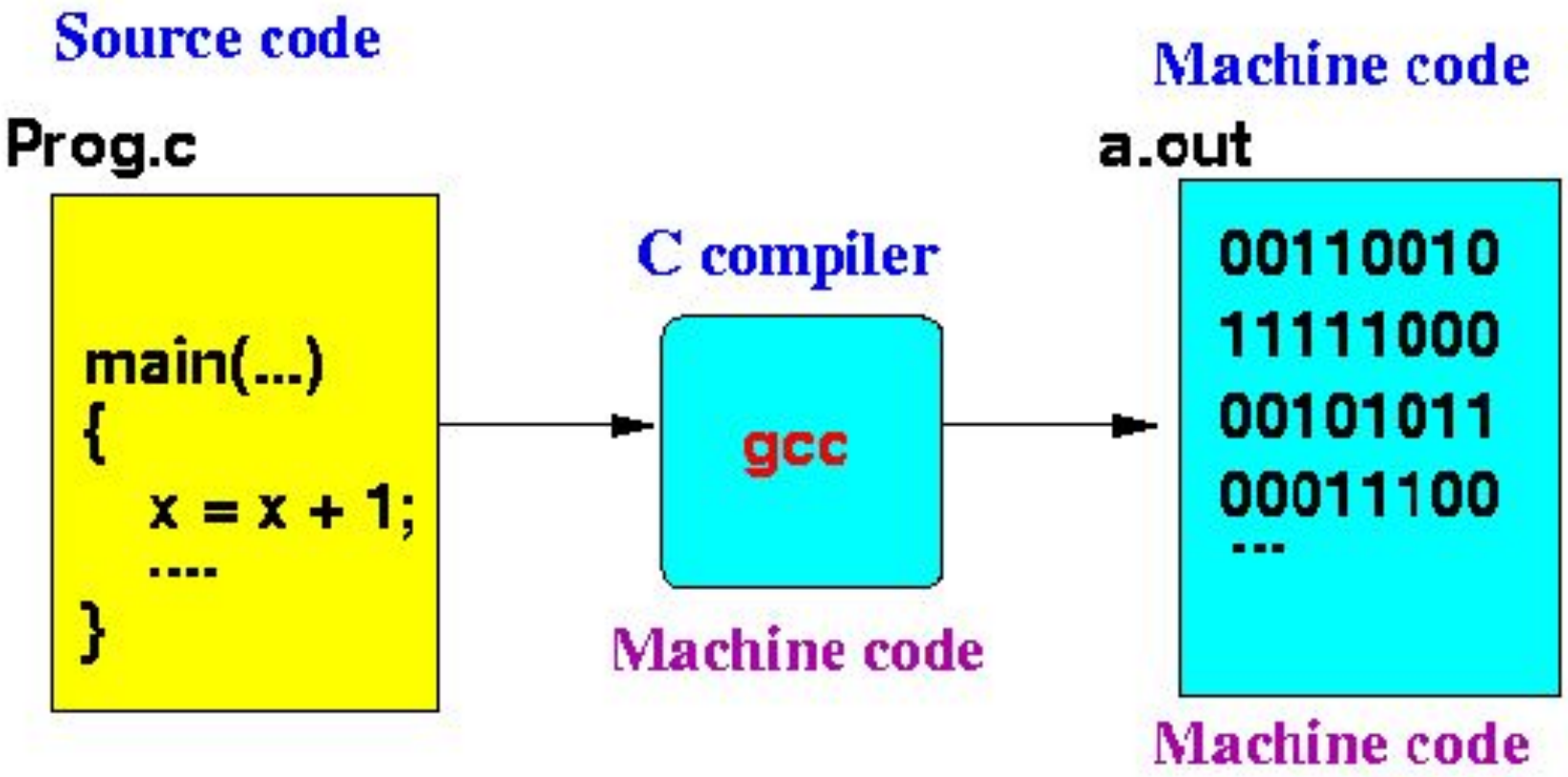
What is Compiler?

- A **compiler** is a tool which has the ability to read the **source code** and translate it to **object level code.**

Why do we need compiler to execute the program?

- Because computer can't understand the **source code** directly. It will understand only **object level code.**
- Source codes are human readable format but the system cannot understand it.
- So, the compiler is intermediate between **human readable** format and **machine-readable format**.
- The **compiler** will parse the source file and translate it into machine understandable object file.




Source code
Prog.c
main(...)
{
x = x + 1;
....
}
C compiler
gcc
Machine code
Machine code
a.out
00110010
11111000
00101011
00011100
...
Machine code

How to Compile C Program in Linux?

1. Open **terminal**

   Use the text editor (gedit)

   Open file using,

   gedit hello.c (file name can be anything but it should end with **dot c** extension) command.

2. To save the file: **Ctrl + S**. It will save the file.
3. To **compile** the program:

   Type,

   gcc hello.c **gcc hello.c -o hello**

4. To **Run** the program:

   Type,

   ./a.out **./hello**

5. To Edit the file: Again type **gedit hello.c**. Edit your program.

You can Compile and Run C Program two ways!

1

Compile the Program

```
gcc hello.c
```

- If there are no errors, this will create an **executable file** with the default name **a.out**

Run the Program

```
./a.out
```

2

Compile the Program

```
gcc hello.c -o hello
```

- Alternatively, you can specify a custom name for the **executable file**.
- The file will be named **hello** instead of **a.out**.

Run the Program

```
./hello
```

Hello World Program in C

Example:

```
#include<stdio.h>      //header section

int main()             //main section
{
    printf("Hello World");
    return 0;
}
```

Output:

Hello World

Input and Output Functions

printf Function (Output)

- When the C program needs to communicate with the user, We should use **printf** function.
- The communication can be,

Greeting the user	**Asking input from the user**
"welcome to c program"	"Enter your username/password" etc.

- **printf** function is defined in stdio.h header file.
- So if we want to use the **printf** function in our c program, we should **include stdio.h** header file.

Syntax:

```
printf("");
```

Example:

```
printf("Welcome to C Programming");
```

- It will print characters to the screen which is given inside the **double quotes " "**
- it should end up with a **semicolon ;**

Contd...

Sample Program:

```c
#include<stdio.h>    //header section

int main()           //main section
{
  printf("Welcome to the C Programming");

  return 0;
}
```

Correct the below Program:

```c
#include<stdio.h>

int main()
{
    printf("Correct me!)

    return 0;
}
```

Contd...

Escape Sequence

- Let's take a scenario where we need to print "Hello" in first line and "World" in the next line.

Example

Hello

World

```
#include<stdio.h>

int main()
{
    printf("Hello");
    printf("World");

    return 0;
}
```

- Do you think the above program will print our expected output? No, it will print **HelloWorld** in the same line as an output. So, how to deal with it. This kind of situation we should go for the **escape sequence.**
- **To print a new line**, we should use **\n** escape sequence.

Contd...

Like Below

```
#include<stdio.h>

int main()
{
    printf("Hello\nWorld");
    return 0;
}
```

\n - New line

Contd...

What is Escape Sequence?

- The sequence of character which has indirect meaning when it placed within double quotes.
- It will optimize some the repetitive tasks while programming.

Example:

While printing some statement, if you want to give horizontal tab (usually four spaces) in between every word like below,

```
#include<stdio.h>
int main()
{
   printf("Happy    New    Year");
   return 0;
}
```

- Here, we manually gave four space between each word.
- This can be achieved easily with **\t** escape sequence.

Contd...

Like Below

```
#include<stdio.h>

int main()
{
    printf("Happy\tNew\tYear");
    return 0;
}
```

\t - Horizontal tab

Task

1. Write a C program to print in the following format:

   Hai

   Welcome

   Today is Holiday

2. Write a C program to print in the following format:

   Hai Welcome

   Today is Holiday

Task

3. Write a C program to print in the following format:

```
*
* *
* * *
* * * *
```

4. Write a C program to print in the following format:

```
* * * *
* * *
* *
*
```